

1. 什么是 RAG ?

RAG (Retrieval-Augmented Generation , 检索增强生成) 是一种结合信息检索与大语言模型生成的技术。

它的核心思想是：先从外部知识库中检索与用户问题相关的文档片段，再把这些片段作为上下文交给大模型生成回答。

这样可以减少大模型的“幻觉”，让回答更基于真实文档内容。

2. RAG 的基本流程

第一步：文档加载 — 读取 PDF、TXT 等格式的本地文档。

第二步：文本切分 — 将长文档切成较小的文本块 (chunk)，便于向量化和检索。

第三步：向量化 — 使用 Embedding 模型将文本块转换为向量表示。

第四步：向量存储 — 将向量存入向量数据库 (如 Chroma)，支持相似度搜索。

第五步：相似度检索 — 用户提问时，将问题也向量化，检索最相关的文本块。

第六步：生成回答 — 将检索到的文本块与问题一起输入大模型，生成最终回答。

3. 为什么需要文本切分？

大模型有上下文长度限制，无法一次性处理整本书或很长的 PDF。

切分后，系统可以精准检索与问题最相关的段落，而不是把整个文档塞给模型。

常见的 chunk size 为 300–1000 字符，overlap 为 50–200 字符。

4. 向量数据库的作用

向量数据库专门用于存储和检索高维向量。

当用户提问时，系统计算问题向量与文档向量之间的相似度 (如余弦相似度)，返回最相关的 top-k 个文本块。

Chroma 是一个轻量级的本地向量数据库，适合学习和原型开发。

5. 如何减少大模型幻觉？

- 只基于检索到的文档内容回答，并在 prompt 中明确要求“如果上下文中没有答案，请说明找不到”。
- 提高检索质量：调整 chunk size、增加 top-k、使用更好的 embedding 模型。

- 返回引用来源，让用户可以核对原始文档。

6. 常见优化方向

- 调整 chunk size 和 overlap
- 换用更强的 embedding 模型
- 增加 rerank (重排序) 步骤
- 对 PDF 做 OCR 或表格解析
- 加入对话历史 (多轮问答)

7. 本项目技术栈

- Python : 编程语言
- LangChain : 大模型应用框架，串联文档加载、切分、检索、生成
- Chroma : 本地向量数据库
- Streamlit : Web 界面
- HuggingFace Embeddings : 本地文本向量化
- OpenAI / Ollama : 大语言模型 (生成回答)

8. 面试常见问题

Q: 你为什么要用 RAG ?

A: 因为大模型无法直接访问用户本地 PDF 的内容，RAG 可以让模型基于私有文档回答，减少幻觉。

Q: RAG 的流程是什么？

A: 文档加载 → 文本切分 → 向量化 → 向量存储 → 相似度检索 → 大模型生成回答。

Q: 如果检索结果不准确，你怎么优化？

A: 可以调整 chunk 大小、增加 overlap、换 embedding 模型、增加 rerank、或改进文档预处理。